Pokemon Sprite Generator

Name: Boodida Pavan Veera Satya Durgesh
Profile: Data Science Intern
Mentor: Teja Maddala
Project Idea: Generating new Pokemon-Style sprites from scratch using the trained Denoising Diffusion Probabilistic Models(DDPM) from the given dataset using diffusers library.
Dataset: huggan/pokemon · Datasets at Hugging Face

Problem Fit:
In image processing, an image can be produced using different techniques like, Diffusion theory, Generative Adversarial Networks(GAN), and Variational Autoencoders(VAE) etc, but diffusion models are best suited for our problem as we have already some of the training images for our model so that, we can be able to manipulate between these images and try to get the best trained model using these images.

In the given pokemon dataset, the diffusion techniques are used because of the less reliance of the GANs. Diffusion techniques produce sharper images with fine details than GANs because there is no mode collapse. GANs use generators and discriminators for the producing of the fine images where they will not work on cooperating, they will work on collapsing the other. This gives reliance on one part of the model on the other. Even if one fails to deliver the problem then the other also collapses. On the other hand Diffusion theories use noisy images to feed and produce fine and sharper images and produce even more sharper images by training the models on different parameters.Also for larger datasets GANs fail to deliver, as we are augmenting the data it might increase the number of training data. So it is better to use Diffusion techniques.

There is a quantitative metric for the differentiation of these images i.e., Fréchet Inception Distance(FID). After training the model using the architecture we need to differentiate between both the batch of images so that there is a clear distinction between the two sets of images. The FID is metric so that it compares the statistical distribution of features between the batch of images.

Scope:

- **Goals:**Train a UNet model with DDPM pipeline with different timesteps and different beta schedules and different ema decay rate

- **Deliverables:**Showing off the Gradio UI setup with seed slider and png export and also adding classifier-free guidance.
- **Tasks and working breakdown:**First, we construct a robust data pipeline to load, augment, and normalize the raw Pokemon sprites into 64x64 tensors ready for neural network ingestion.Second, I executed the core Hyperparameter tuning by independently training a UNet using Apple Silicon-optimized mixed precision, gradient accumulation, and custom noise schedules. Finally, we evaluate the trained models side-by-side by computing the quantitative Fréchet Inception Distance (FID) score and generating visual comparison grids to declare a definitive winner.
- **OUT of Scope:**This project has certain out of scope things that can be done so that it makes the project somewhat interesting like, Text-to-Image generation,High resolution upscaling,A/B Testing like comparing two models on the same parameters.
- **Constraints:**We have been restricted on some of the ideas like to run locally only on the MAC chip,dataset size of 7,300+ images as diffusion models use millions of images for the training and we are also restricted to the number of epochs training like only to 50.

<u>Data Engineering and Pipeline:</u>

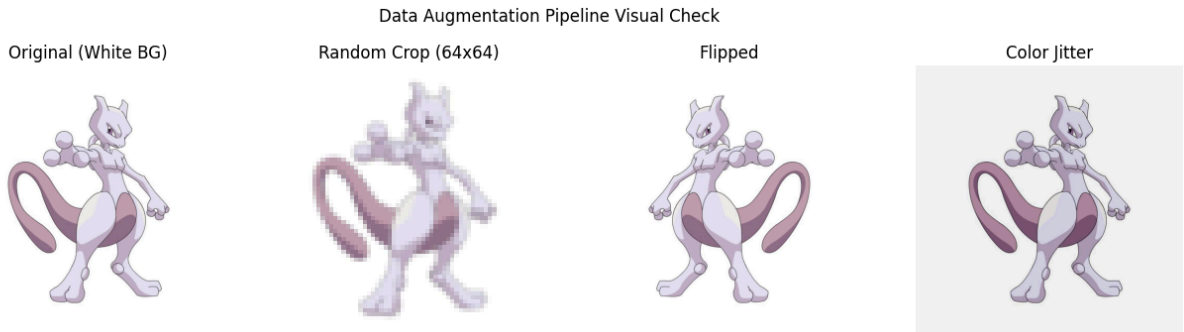
Libraries and its Modules-os (for file storing and saving trained models), numpy (for mathematical operations), matplotlib (uses its module pyplot for visualizing statistics and image), datasets (loads datasets directly from the Hugging face platform), torch (building neural networks and for training them), torch.nn.functional (for using activation functions and loss functions), torch.utils.data (for loading datasets in batches during training), torchvision (uses its module transforms for data augmentations and transformations and it's one module called utils is used for combining images into one grid), diffusers (used for importing various neural network builtins so that no need to write those from scratch like UNet,DDPM Scheduler), diffusers.training_utils (for stabilizing the training and improving the generated image quality), tqdm (Used for tracking the progress of the epoch training), accelerate (handles device management and distributed training), gradio (for using the builtin web UI called Gradio for png results and exporting).

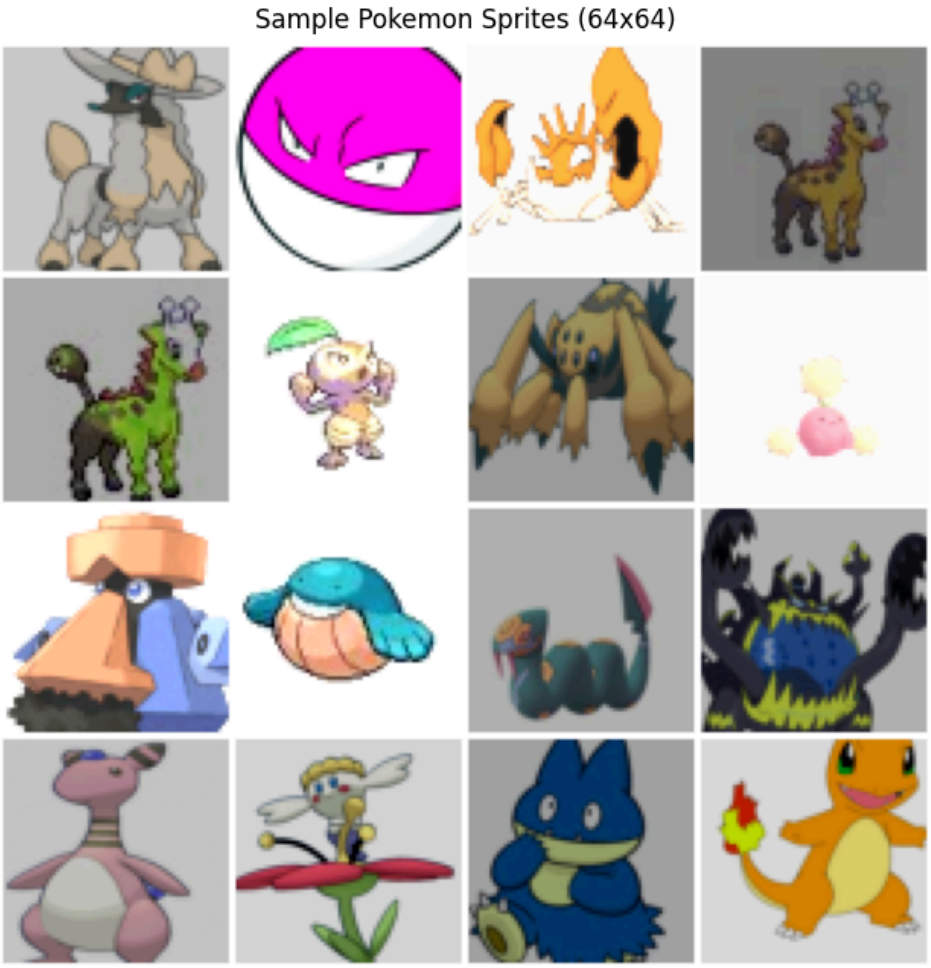
Workflow:

<u>1.Dataset Loading and Data Augmentations:</u>All the images will be of different resolutions,different backgrounds,different brightness, and also of different RGB

values.As UNet trains on the data which is of uniformity it can be able follow its network architecture internally.

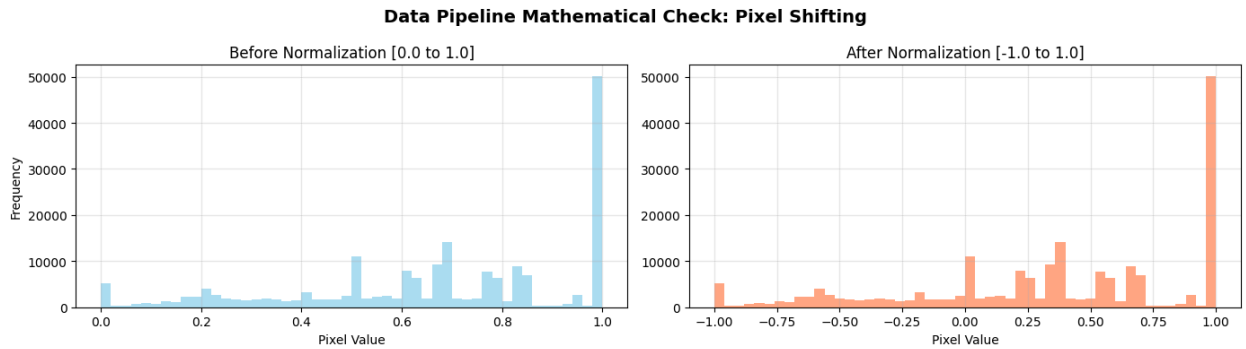
After preprocessing we got some visualizations,





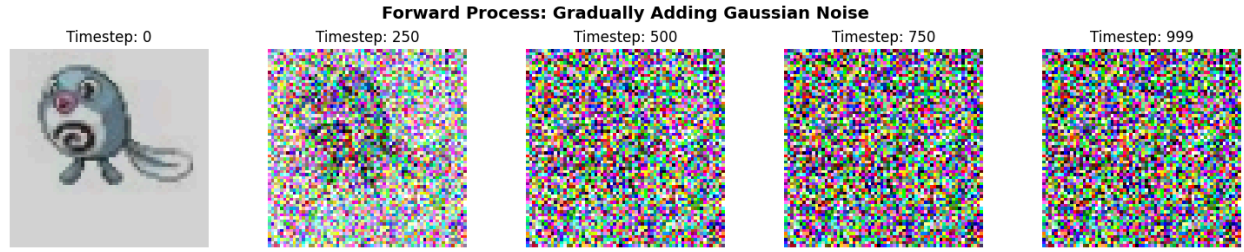
The above image is the representation of a batch of images if taken for the evaluation of the model for the Frechet Inception Distance(FID).

The below image is the histogram for the checking of the pixel value distribution before and after the normalization.



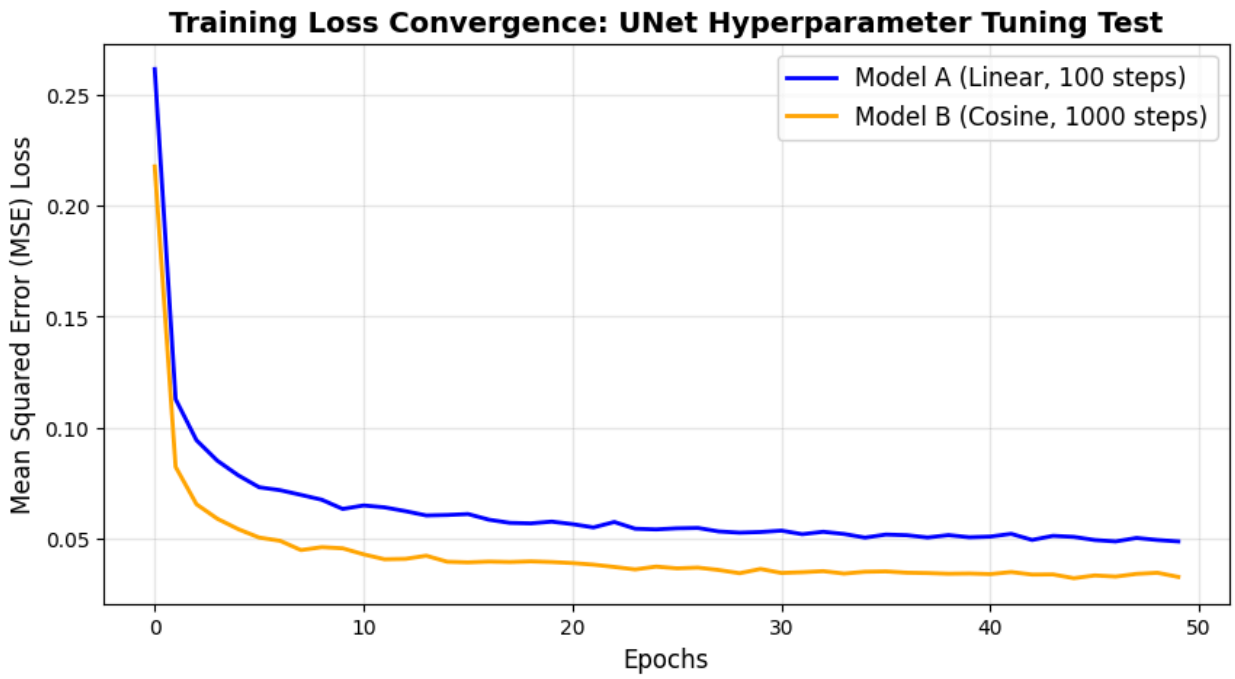
<u>2.The Core Architecture(UNet):</u>Once the perfectly uniform 64x64 images leave the DataLoader,they are fed into the UNet. At a high level,the UNet acts as the "brain" of the pipeline.Its only job is to look at a severely corrupted, noisy image(along with a timestamp of how noisy it is) and guess exactly what the noise looks like.It doesn't generate Pokemon directly instead,it generates a *noise prediction* map that tells the system exactly what to subtract from the image to make it look slightly more like a real Pokemon.

<u>3.The Noise Scheduler(DDPM):</u>The noise scheduler acts as the strict timekeeper for the entire training process. During training, it is responsible for the "forward process": mathematically destroying the perfect Pokemon images by injecting specific, calculated amounts of random static (noise) based on a specific timestep(e.g., step 100 vs step 1000).It ensures that the UNet learns to denoise images at every possible stage of destruction, from slightly fuzzy to completely static.



The above image is the picture of one particular image in a dataset progression from the addition of the noise at different timesteps.More timesteps means more fine details can be reconstructured.

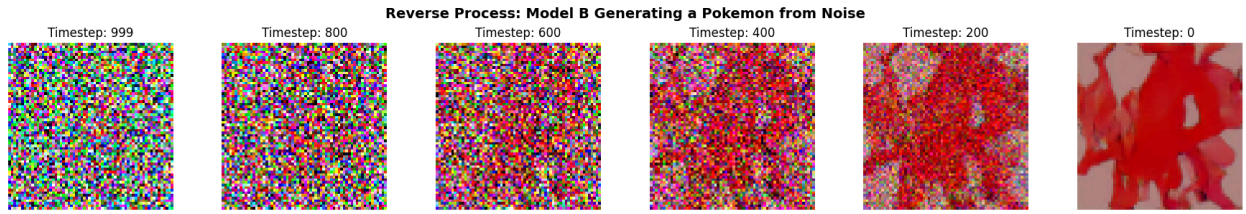
4.The Loss Function(F.mse_loss): This is the mathematical grader for the UNet. After the DDPM scheduler adds real noise to an image, and the UNet makes its guess about what that noise looks like, the Loss Function compares the two. By using Mean Squared Error (MSE), it calculates the exact mathematical distance between the UNet's *predicted* noise and the *actual* noise. This resulting "loss" score is the critical signal sent back into the model to update its weights so it can guess better next time.



5. Hardware Management (Accelerator): Deep learning requires a massive amount of memory and computational routing.The Accelerator acts as the traffic controller between the Python code and the Mac's physical hardware. Instead of writing complex, manual code to move every single tensor to the Apple Silicon GPU (MPS), manage float16 precision, and handle gradient accumulation, the Accelerator intercepts the math from the Loss Function and safely distributes the workload so that the computer doesn't crash from memory overload.

6. Training Loop Monitoring (tqdm): Training a diffusion model requires churning through thousands of batches, tqdm acts as the real-time dashboard. By wrapping up the DataLoader, it hooks into the training loop to calculate exactly how many batches are left, estimates the time remaining, and dynamically prints the current Loss score from Step 4 directly to the screen. This proves that the pipeline is actually learning without worrying about the model being trained. It shows the progress for each epoch in the form of percentage.

Denoising the image for reconstruction: After denoising the image, it can be observed that more the number of timesteps then more can be the finer image reconstructed from the noise so that we can be able to get the exact desired FID score



Parameters for training different models: While training a diffusion model using a U-Net architecture with a DDPM scheduler, I encountered several parameters that significantly influence how the model learns and how the generated images behave. Among the most important are timesteps, EMA decay rate, and the beta schedule used in the diffusion process. Understanding how these parameters work is essential because they affect training stability, image quality, computational cost, and overall model behavior.

1. **Timesteps:**Timesteps represent the number of stages through which noise is gradually added to an image and then learned to be removed by the model.Timesteps define how fine-grained the image is after using the neural network. Increasing timesteps increases the fine details restoration, smooth diffusion, better reconstruction of an image. But it takes more computational cost, larger computational load, slower training.

2. **EMA(Exponential Moving Average) Decay Rate:** During training, model weights constantly change as gradients update them. These updates can sometimes introduce instability or fluctuations.EMA maintains a smoothed version of the model weights by gradually averaging them over time. A high decay value helps produce stable and consistent outputs, which is especially important for generative models like diffusion networks.
3. **Beta Schedule(Cosine vs Linear):** In diffusion models, beta controls the amount of noise added at each timestep. If noise grows too fast,the model struggles to learn the denoising process.If noise grows too slowly,training may become inefficient.So choosing the correct schedule is critical for stable learning. Linear Schedule follows a linear noise addition and is simple,stable and used in early DDPM models. The cosine schedule follows a cosine curve, which

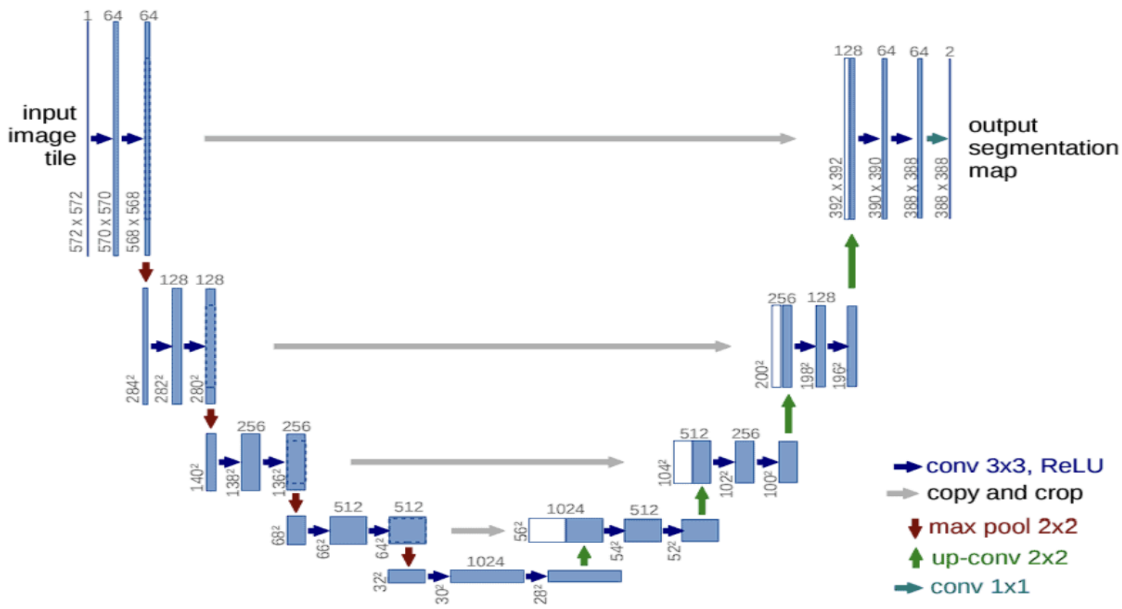
distributes noise more smoothly and has better training stability,improved sample quality, and better preservation of image structure but complex in nature.

4. **DDPM Scheduler:** DDPM scheduler controls how noise is added and removed during the diffusion process.The scheduler defines number of timesteps, betaschedule,noise variance and sampling strategy.

Architecture:

Why choose U-Net?
U-Net is mainly discovered for the biomedical image segmentation.Before the advanced deep learning models like U-Net and Mask R-CNN, image segmentation faced several significant challenges.

- **Precision and Accuracy:** Traditional methods struggled with accurately segmenting images, especially those with varying textures, complex structures, or noise. Researchers used techniques such as thresholding, region-based segmentation, and edge detection; however, these methods did not deliver the necessary detail and accuracy for complex applications like medical imaging.
- **Dependence on Handcrafted Features:** Earlier approaches relied heavily on manual feature extraction methods, which were time-consuming and less robust.
- **Scalability and Efficiency:** Due to handcrafted feature extractors, it was nearly impossible to scale the model, as doing so would require one to handcraft all the features required for the image variations.



Source-https://arxiv.org/pdf/1505.04597v1

The model features a distinctive U-shaped structure, comprising two main parts: the contracting path (encoder) and the expanding path (decoder). The encoding path captures context, and the decoding path enables precise localization.

Contracting Path (Encoder)

Here are the different layers and components in U-Net:

- **Convolutional Layers:** Convolutional layers are the primary components of the contracting path. In the originally proposed model, each block consists of two consecutive 3×3 convolutional layers followed by a Rectified Linear Unit (ReLU) activation function. By stacking multiple convolutional layers, U-Net learns increasingly complex features.
- **Activation Functions:** After each convolution operation, a ReLU activation function is applied. The role of ReLU here is crucial as it introduces

non-linearities into the system, which allows for learning more complex patterns in data that are not possible with just linear transformations.

- **Max Pooling:** Following the convolutional layers, a 2×2 max pooling operation with a stride of 2 is used. This step reduces the spatial dimensions by half. However, it captures abstract information (that makes the model invariant to small shifts and distortions).
- **Feature Doubling:** After each max pooling step, the subsequent convolutional layer doubles the number of filters used. For example, if a layer starts with 64 feature channels, it will have 128 channels after the next pooling and convolution operations. By doubling the number of feature channels, the network can maintain or even increase its capacity to represent information despite the reduction in spatial resolution. This is crucial because the risk of losing important details increases as the image size reduces.

Expansive Path (Decoder)

Aims to recover spatial information and generate the segmentation map using up-convolution (or transposed convolution).

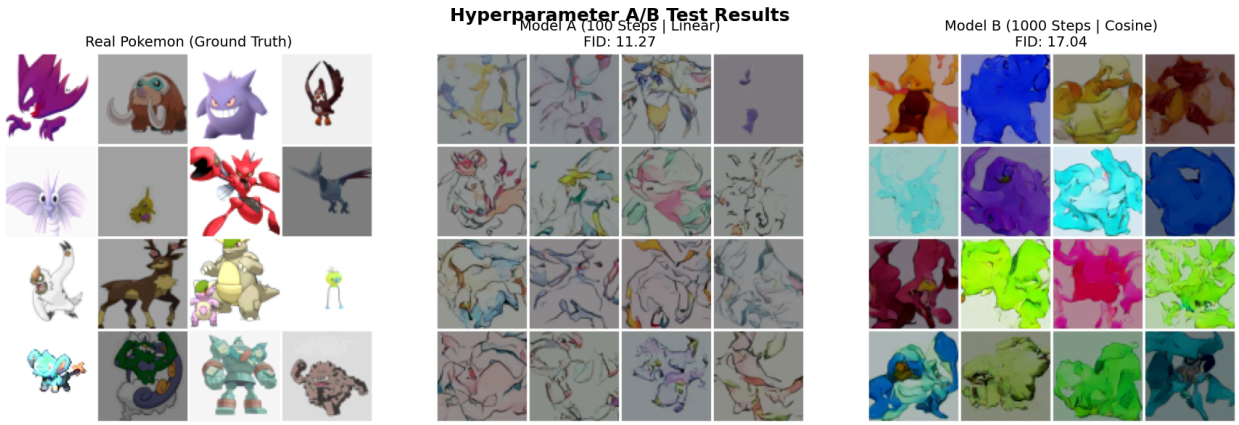
Each block includes:Upsampling of the feature map to increase image size, 2×2 convolution to halve the number of feature channels,Two 3×3 convolutions followed by ReLU activation.
U-Net also uses skip connections.

Skip Connections
Skip connections significantly contribute to U-Net's effectiveness. By merging feature maps from the contracting path directly with the expanding path, U-Net combines low-level detail information with high-level contextual information across the network.

- **Recover Spatial Hierarchies:** These connections allow U-Net to concatenate high-resolution features from the contracting path with up-sampled outputs from the expanding path. This helps recover spatial hierarchies lost during pooling operations in the contracting phase.

Output of U-Net

At the final layer of the Convolutional Neural Network, a 1×1 convolution maps the feature vector (typically with 64 components at the last stage of the expansive path) to the desired number of classes for segmentation.



<u>Evaluation metrics:</u>

FID:Frechet Inception Distance, as discussed earlier should be less than 50 for better image generation.

Under 15: Photorealistic, industry-level production quality.

30 - 50: High quality. Shapes are nearly perfect, maybe a few weird artifacts.

50 - 100: The "Promising Blob" phase. The model understands colors and general shapes, but they might look like melted ice cream.

100+: Mostly colorful static or severe distortions.

As we got the,

FID score using Model-A:11.27

FID score using Model-B:17.04

For the project requirement the considered FID score should be less than 50, but the model is having 11.27.Generally, Model-B must be having the less FID but I think due to the less number of epochs it couldn't train much.

So model-A is more suitable for using the model for gradio purposes.

The Gradio UI using the model-A is generated using this link,

https://482a2724ef724a5f00.gradio.live

Conclusion:

Looking back at this project, building a custom diffusion model from scratch was one of the most challenging but rewarding things I've coded. I didn't just want to pull an off-the-shelf model; I wanted to build the math, process the raw pixels, and actually understand how a UNet hallucinates a Pokémon out of pure static.Training this locally on my Mac's Apple Silicon (MPS) was a journey. Between fixing the pure-black background bugs in the image tensors and surviving an absolute dependency hell between Pydantic, FastAPI, and Gradio at the very end, I definitely earned my deployment stripes.

But the biggest takeaway for me was the A/B test. Going into this, I fully expected Model B (1000 inference steps with a Cosine schedule) to completely outclass Model A (100 steps with a Linear schedule). In theory, more steps and a smoother noise curve should equal a better image.Instead, Model A absolutely crushed it, hitting a highly realistic FID score of 11.27

It taught me a massive lesson about compute constraints: because I was only training for 50 epochs, the aggressive, blocky noise removal of the Linear schedule actually *forced* the model to learn bold shapes and vibrant colors much faster. The Cosine model was too gentle, and just didn't have the runway (enough epochs) to reach its full potential. It was a great reminder that in machine learning, the "fancier" hyperparameter isn't always the right choice for the specific hardware and time constraints.

Next Experiments:

1. Letting the Cosine Model "Cook":Now I know that 50 epochs isn't enough for a Cosine schedule. My next experiment is to leave my Mac running overnight, push Model B to 300 or 500 epochs, and see if that slow-burn training finally surpasses the Linear model's fidelity. I want to see how smooth those textures can actually get.

2. Adding Class Conditioning (Type-to-Image):Right now, my Gradio app takes a random seed and generates a totally random Pokémon. The next architectural step is to add an Embedding layer to the UNet so I can pass in text labels. I want to be able to hit a dropdown menu, select "Fire/Dragon", and have the model deliberately generate a custom sprite that fits that elemental typing.

3. Permanent Cloud Deployment:I successfully got the Gradio app running locally and bypassing my corporate proxy using temporary `.live` links. The next engineering step is to wrap this entire application in a Docker container and push it to Hugging Face Spaces or AWS. I want to turn this from a local script into a permanent web app I can drop in my portfolio.